

武器の追従

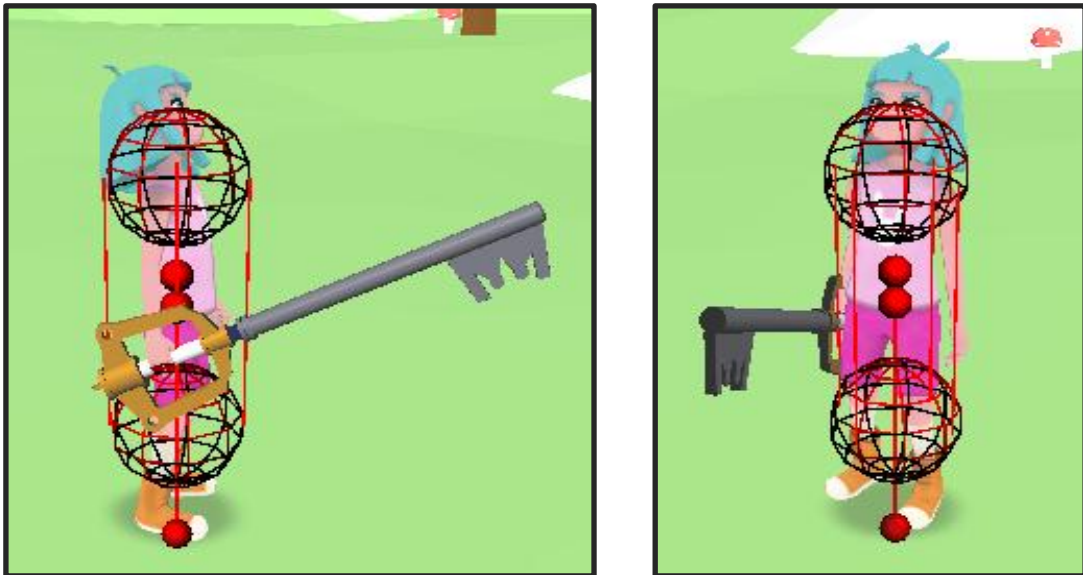
キャラクターの手に武器を持たせる場合、

- ① BlenderなどのCGソフトで、予め武器をキャラクターに持たせる
- ② プログラムで武器モデルを追従、同期させる

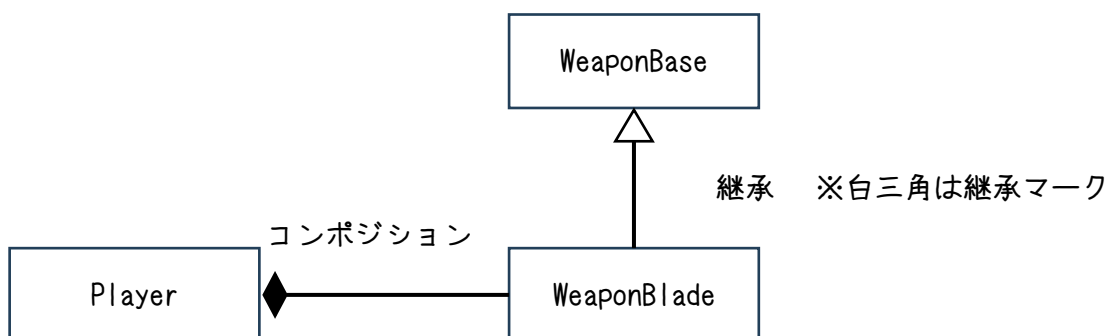
上記の2パターンがあります。

DxLibの仕組み上、①の方がかなり処理速度が早いのですが、ダウンロードしてきたキャラクターモデルを、Blenderなどで読み込ませてしまうと、モデルやボーンが崩れてしまうことがよくあります。

そうなってしまった場合、3DCGの専門職ではない私たちには対処が難しいので、ここでは、プログラマーらしく②の方法で実装していきます。



クラス図



WeaponBase.h

```
#pragma once
#include <DxLib.h>
#include "../ActorBase.h"
class Transform;

class WeaponBase : public ActorBase
{
public:

    // コンストラクタ
    WeaponBase(const Transform& followTransform, int followFrameId);

    // デストラクタ
    virtual ~WeaponBase(void) override;

    // 更新
    virtual void Update(void) override;

protected:

    // 追従先Transform
    const Transform& followTransform_;

    // 追従対象のフレームID
    int followFrameId_;

    // ローカル座標
    VECTOR localPos_;

    // ローカル回転
    VECTOR localRot_;

};
```

WeaponBase.cpp

```
#include "../../Utility/AsoUtility.h"
```

```

#include "../..../Utility/ModelFrameUtility.h"
#include "../..../Common/Transform.h"
#include "WeaponBase.h"

WeaponBase::WeaponBase(const Transform& followTransform, int followFrameId)
:
    followTransform_(followTransform),
    followFrameId_(followFrameId),
    localPos_(AsoUtility::VECTOR_ZERO),
    localRot_(AsoUtility::VECTOR_ZERO)
{
}

WeaponBase::~WeaponBase(void)
{
}

void WeaponBase::Update(void)
{
    // 対象フレームの位置にtargetを配置し、
    // 対象フレームの回転に加え、指定した相対座標・回転を加える
    ModelFrameUtility::SetFrameWorldMatrix(
        followTransform_, followFrameId_, transform_,
        localPos_, localRot_
    );

    // 上記関数内で更新された行列情報からクォータニオンに変換
    transform_.quaRot = Quaternion::GetRotation(transform_.matRot);

    // 大きさ、回転(クォータニオン)、座標を元にモデルを更新
    transform_.Update();
}

```

```

WeaponBlade.h

```

```

#pragma once
#include <DxLib.h>
#include "WeaponBase.h"
class Transform;

```

```

class WeaponBlade : public WeaponBase
{

public:

    // コンストラクタ
    WeaponBlade(const Transform& followTransform, int followFrameId);

    // デストラクタ
    ~WeaponBlade(void) override;

    // 更新
    void Update(void) override;

protected:

    // リソースロード
    void InitLoad(void) override;

    // 大きさ、回転、座標の初期化
    void InitTransform(void) override;

    // 衝突判定の初期化
    void InitCollider(void) override;

    // アニメーションの初期化
    void InitAnimation(void) override;

    // 初期化後の個別処理
    void InitPost(void) override;

private :

    // モデルの大きさ
    static constexpr float SCALE = 0.2f;

    // 衝突判定用カプセル上部球体
    static constexpr VECTOR COL_CAPSULE_TOP_LOCAL_POS = { 0.0f, 130.0f, 0.0f };

    // 衝突判定用カプセル下部球体

```

```

static constexpr VECTOR COL_CAPSULE_DOWN_LOCAL_POS = { 0.0f, 20.0f, 0.0f };

// 衝突判定用カプセル球体半径
static constexpr float COL_CAPSULE_RADIUS = 30.0f;

};

```

WeaponBlade.cpp

```

#include "../../Utility/AsoUtility.h"
#include "../../Manager/ResourceManager.h"
#include "WeaponBlade.h"

WeaponBlade::WeaponBlade(const Transform& followTransform, int followFrameId)
:
    WeaponBase(followTransform, followFrameId)
{
}

WeaponBlade::~WeaponBlade(void)
{
}

void WeaponBlade::Update(void)
{
    WeaponBase::Update();
}

void WeaponBlade::InitLoad(void)
{
    // モデルのロード
    transform_.SetModel(
        resMng_.Load(ResourceManager::SRC::WEAPON_BLADE).handleId_);
}

void WeaponBlade::InitTransform(void)
{
}

```

```

transform_.scl = VScale(AsoUtility::VECTOR_ONE, SCALE);
transform_.quaRot = Quaternion();
transform_.quaRotLocal = Quaternion();
transform_.pos = AsoUtility::VECTOR_ZERO;

localPos_ = { -2.0f, 0.0f, -3.0f };
localRot_ = {
    AsoUtility::Deg2RadF(0.0f),
    AsoUtility::Deg2RadF(0.0f),
    AsoUtility::Deg2RadF(-90.0f)
};
}

void WeaponBlade::InitCollider(void)
{
}

void WeaponBlade::InitAnimation(void)
{
}

void WeaponBlade::InitPost(void)
{
}

```

ModelFrameUtility.h

```

#pragma once
#include <DxLib.h>
class Transform;

class ModelFrameUtility
{
public:

#pragma region ワールド系

```

```

// 対象フレームのワールド行列を大きさ・回転・位置に分解してを取得する

```

```

static void GetFrameWorldMatrix(
    int modelId, int frameIdx, VECTOR& scl, MATRIX& matRot, VECTOR& pos);

// 対象フレームの位置にtargetを配置し、
// 対象フレームの回転に加え、指定した相対座標・回転を加える
static void SetFrameWorldMatrix(
    const Transform& follow, int followFrameIdx,
    Transform& target, VECTOR localPos, VECTOR localRot);

#pragma endregion

};

```

ModelFrameUtility.cpp

```

#include "../Utility/AsoUtility.h"
#include "../Object/Common/Transform.h"
#include "ModelFrameUtility.h"

void ModelFrameUtility::GetFrameWorldMatrix(
    int modelId, int frameIdx, VECTOR& scl, MATRIX& matRot, VECTOR& pos)
{
    // 対象フレームのローカル座標からワールド座標に変換する行列を得る
    // ( 大きさ、回転、位置 )
    auto mat = MVIGetFrameLocalWorldMatrix(modelId, frameIdx);

    // 拡大縮小成分
    scl = MGetSize(mat);

    // 回転成分+拡大縮小成分
    matRot = MGetRotElem(mat);

    // 回転成分のみにする
    auto revScl = VGet(1.0f / scl.x, 1.0f / scl.y, 1.0f / scl.z);
    matRot = MMult(matRot, MGetScale(revScl));

    // 移動成分
    pos = MGetTranslateElem(mat);
}

```

```

void ModelFrameUtility::SetFrameWorldMatrix(
    const Transform& follow, int followFrameIdx,
    Transform& target, VECTOR localPos, VECTOR localRot)
{

    // 対象フレームのローカル座標からワールド座標に変換する行列を得る
    // ( 大きさ、回転、位置 )
    MATRIX worldMatMix =
        MVIGetFrameLocalWorldMatrix(follow.modelId, followFrameIdx);

    ○○○
    ( 30行~50行くらい )

}

```

【要件】

PlayerクラスにWeaponBladeを実装し、
プレイヤーの手に武器モデルを描画すること。

- ・ 右手のフレーム番号は44がお勧め
- ・ 武器モデルは何でも良いが、
TeamsのDataフォルダにあるWeaponを使用しても良い
- ・ SetFrameWorldMatrix関数の完成

非常に難易度が高いですが、3D計算の基礎部分にあたるため、
最終、できなくても、チャレンジして欲しいです。
DxLibを使用しないDirectX使い、パブリッシャー、大手狙いの学生さんは、
当然、このあたりは熟知しています。

MVIGetFrameLocalWorldMatrixは、
対象フレームのワールド合成行列を取得できる関数です。
(ローカル行列ではない)

その行列にローカル回転やローカル座標を合成すると、
手の回転や位置に合わせつつ、引きモデルの回転や位置を微調整できます。

しかし、合成行列(大きさ、回転、位置)のまま、
行列合成を行ってしまうと、不要な情報も合成されるため、
正しい合成になりません。(回転合成だったら、位置、大きさは不要)

合成行列から大きさ成分を抜き出す

MGetSize

合成行列から大きさと回転成分を抜き出す

MGetRotElem

※ DxLibのヘルパーには、
回転成分を抜き出すと記載があるが、大きさも含まれる

大きさを除去するには、スケール倍を元に戻す

1.0f / scl

合成行列から移動成分を抜き出す

MGetTranslateElem

※今回は使用しなくても実装可

(1.0f, 0.0f, 0.0f)は、ワールドのX軸であり、右手のX軸ではない
任意軸の回転は、MGetRotAxis関数を使用

【目標】

上記のやりかたではなくとも良いですが、
プレイヤーの右手に違和感なく武器が装備されていること。